Computation for Analytics

DIANE WOODBRIDGE, PH.D

Content

Object Oriented Programming

- Class
  - Attributes
  - Methods
- Object
- Class Inheritance

The Big Reveal



We've been working with functions and modules with functions, as well as defining our own functions.

We've also been working with class objects all along, we just haven't recognized them as such!

```
x = 'Hi'
print( x.lower() ) # send message lower to x
print( type(x) )
print( type(x.lower) )
```

```
hi
<class 'str'>
<class 'builtin_function_or_method'>
```

Content

Object Oriented Programming

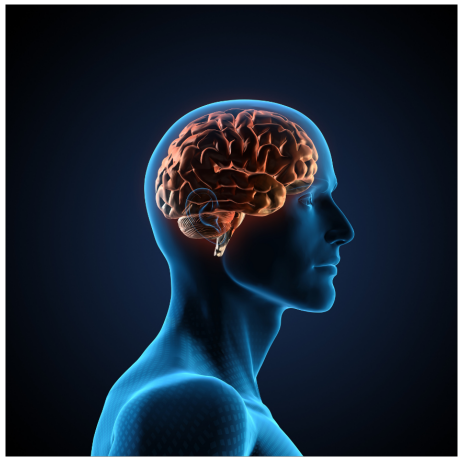
- Class
  - Attributes
  - Methods
- Object
- Class Inheritance

Object Oriented Programming (OOP)

A human views the world by abstracting all the objects with common properties.

- Example.
  - Diane is a **human** that has a name, eats, and sleeps.

An OOP paradigm maps the programming to real-world objects.



https://en.wikipedia.org/wiki/Object-oriented_programming

Object Oriented Programming (OOP)

Without OOP

- There's no template that ensures each dictionary has the right key/value pairs (actually, Python syntax has no way to do this)
- The notation is a bit awkward: diane['firstname'] instead of diane.firstname
- There's no way to associate functions with these dictionaries

```
diane = {"firstname" : "Diane", "lastname" : "Woodbridge"}
```

```
diane["firstname"]
```

```
'Diane'
```

https://en.wikipedia.org/wiki/Object-oriented_programming

Content

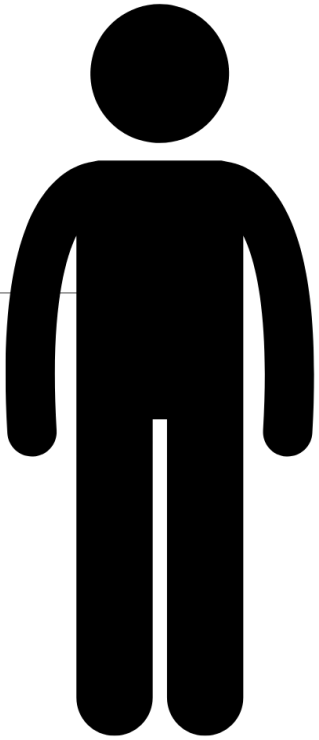
Object Oriented Programming

- **Class**
  - Attributes
  - Methods
- Object
- Class Inheritance

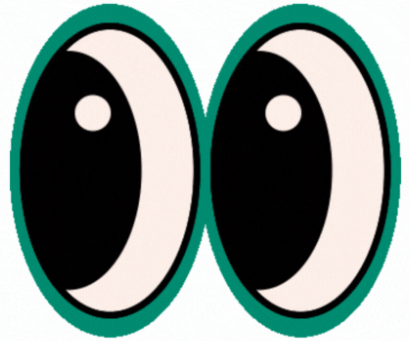
Class

A **class** is the blueprint/prototype for an object.

- It can have **attibributes**, which are variables belong the class and represent states.



- It can have **methods**, which are functions that represents its behaviors.



Content

Object Oriented Programming

- Class
  - Attributes
  - Methods
- **Object**
- Class Inheritance

Class vs Object

A **class** is the blueprint/prototype for an object.

- It can have **attibributes**, which are variables belong the class and represent states.
- It can have **methods**, which are functions that represents its behaviors.

An **object** is an instance of **class.**

- You can create an object by initializing a class and providing attribute values.



Class

You can create class using keyword `class`

- Define a **class**' constructor which can initiate the object with attributes and its methods.

```
class ClassName( ):
    def __init__(self, val1, val2):
        self.attr_1 = val1
        self.attr_2 = val2
    def method1(self):
        statements
```

Class methods **require** an extra first parameter `self`

You can instantiate an **object** by calling

```
object = ClassName(val1, val2)
```

- You can call attributes by `object.attr_1` and methods by `object.method1()`

Class

You can create class using keyword `class`

- Define a **class**' constructor which can initiate the object with attributes and its methods.

```
class ClassName( ):
    def __init__(self, val1, val2):
        self.attr_1 = val1
        self.attr_2 = val2
    def method1(self):
        statements
```

__init__ is a constructor and this is first being executed as soon as an object is instantiated.

You can instantiate an **object** by calling

```
object = ClassName(val1, val2)
```

- You can call attributes by `object.attr_1` and methods by `object.method1()`

Class

You can create class using keyword `class`

- Define a **class**' constructor which can initiate the object with attributes and its methods.

```
class ClassName( ):
    def __init__(self, val1, val2):
        self.attr_1 = val1
        self.attr_2 = val2
    def method1(self):
        statements
```

```
1 class Human():
2     def __init__(self, first_val, last_val):
3         self.firstname = first_val
4         self.lastname = last_val
5     def introduce(self):
6         print(f'My name is {self.firstname} {self.lastname }.')
```

✓ 0.0s

```
1 curry = Human("Stephen", "Curry")
```

✓ 0.0s

```
1 curry.introduce()
```

✓ 0.0s

```
My name is Stephen Curry.
```

You can instantiate an **object** by calling

```
object = ClassName(val1, val2)
```

- You can call attributes by `object.attr_1` and methods by `object.method1()`

ex01-oop-human.ipynb

Python Naming Styles and Class

We will learn about PEP8 (Style Guide for Python Code)

- I'd like to cover some of them to understand why names of some variables/ attributes/methods/functions start or end with underscores (_)
- __double_leading_underscore: when naming a ***class*** attribute, invokes name mangling and Python interpreter changes it to _className__double_leading_underscore
- __double_leading_and_trailing_underscore__: "magic" attributes or methods which generally used for built-in attributes/methods (Ex. __init__, __add__)
- _single_leading_underscore: weak private (internal use only) variable indicator.
- single_trailing_underscore_: Use to avoid conflicts with Python keyword (Ex. class_)

https://pep8.org/#descriptive-naming-styles

Example 2

Create a function called, create_station_object_list(file_name) which reads data from file_name (.csv) and creates a list of EpaStation objects which includes name, station_id, state, county, lon, lat and aqi_record array of an EPA station.

- EpaStation Class - its constructor takes/assigns name, station_id, state, county, lon, and lat. This class also has aqi_record which is a list of dictionary in a format of {'YYYY-MM-DD': (pm10, aqi)}.
  - You need to Override __eq__ which is invoked when == is being called.
  - Add a method called add_aqi_record(self, date, pm10, aqi), which appends {date:(pm10, aqi)} if the date is not yet in aqi_record.



ex02-epa.ipynb

Content

Object Oriented Programming

- Class
  - Attributes
  - Methods
- Object
- **Class Inheritance**

Inheritance

Defining something new as it relates to something we already know is usually a lot easier than starting from scratch; same is true in programming.

- Ex Student/Engineer/Teacher/Baby are all Human and have Human's characteristics.



Inheritance

A subclass inherits attributes and methods from a superclass

- Inheritance behaves like an import or include operation from another class into a new class (not exactly true)
- Syntax :

```
class Subclass(Superclass):
    def __init__(self, attributes_for_super, attributes_for_sub):
        super().__init__(attributes_for_super)
        self.attribute = attribute
    def method1():
        statements
```

super() is indicating the parent (superclass) of the current class

Inheritance

A subclass inherits attributes and methods from a superclass

- Inheritance behaves like an import or include operation from another class into a new class (not exactly true)
- Syntax :

```
class Subclass(Superclass):
    def __init__(self, attributes_for_super, attributes_for_sub):
        super().__init__(attributes_for_super)
        self.attribute = attribute
    def method1():
        statements
```

You can write this as
self.super_class_attr1 = val1
self.super_class_attr2 = val2

Inheritance

A subclass inherits attributes and methods from a superclass

- Inheritance behaves like an import or include operation from another class into a new class (not exactly true)
- Syntax :

```
class Subclass(Superclass):
    def __init__(self, attributes_for_super, attributes_for_sub):
        super().__init__(attributes_for_super)
        self.attribute = attribute
    def method1():
        statements
```

This Subclass also includes all the other variables and methods from Superclass.
You can also define additional variables/methods or override them.

Example 3

Student is a subclass of Human and can be initiated with first name, last name and school.

- Create Student class and instantiate an object.
- Does the object have .introduce() method?
- Can we override .introduce() to print 'My name is firstname lastname. I am a student at schoolname.'?

Example 4

Create a class called JumpList which inherits from list.

The constructor for this class takes ***data*** and ***step***, but for its iteration, it iterates data in every "***step***" step.

- Tips
  - Make sure to override the “__iter__", "__next__" method for iteration.
  - You can find more details of list by help(list)
- Example. For jump_list = JumpList([1,2,3,4,5], 3), jump_list returns [1, 4]

```
1 help(list)
```

✓ 0.0s

```
Help on class list in module builtins:

class list(object)
 |  list(iterable=(), /)
 |
 |  Built-in mutable sequence.
 |
 |  If no argument is given, the constructor creates a new empty list.
 |  The argument must be an iterable if specified.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
 |
 |  __contains__(self, key, /)
 |      Return key in self.
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
```

References

Python : https://docs.python.org/3/

Python Classes : https://docs.python.org/3/tutorial/classes.html